



**Tshwane University
of Technology**

We empower people

**FORMAL SUMMATIVE ASSESSMENT
INSTRUCTIONS:**

1. All rules and regulations regarding student conduct and behaviour, as agreed upon by students at registration and as otherwise stated by the Tshwane University of Technology, apply.
2. A student card or proof of registration, along with an original identification document, must be presented to invigilators.
3. Ensure that your student number is correctly indicated on all submissions (paper-based or online) and that the attendance record is signed as required.
4. If a student is suspected of any form of cheating or plagiarism, either during the assessment or afterwards, the examiner, in their professional judgment and after consultation with the appointed moderator, may, at minimum, award a zero mark for the assessment. Further action may be taken against the student.

**DEPARTMENT OF
ELECTRICAL ENGINEERING**

**FACULTY OF
ENGINEERING AND THE BUILT
ENVIRONMENT**

**MODULE CODE
ES216AB**

**MODULE NAME
ENGINEERING SOFTWARE DESIGN**

**ASSESSMENT NAME
Evaluation 3**

EXAMINER:	Mr M. Chauke Prof J.A. Jordaan	MARKS:	65 points
MODERATOR:	Mr T.E. Olivier	PAGES:	15 (incl. cover)
DATE:	18 MAY 2026	TIME:	2 hours

STUDENT NUMBER:								
SURNAME:			INITIALS:			SIGNATURE:		

NOTE THE FOLLOWING:

- 1. Plagiarism Policy:** Original work is required. We will use similarity detection software to review all student submissions for plagiarism. Ensure your work is your own.
- 2. Internet Protocol (IP) Tracking:** IP addresses will be recorded and checked to verify that you have uploaded your work from the correct TUT laboratory.
- 3. No External Devices:** The use of USBs, cell phones or other external devices is prohibited during the evaluation.
- 4. Internet Access:** External internet access is not permitted.
- 5. Evaluation Content:** This evaluation will cover topics from Unit 1 to Unit 6
- 6. Programming Language:** Write your program in C, adhering to structured programming principles.
- 7. Editing Requirements:** Your program must comply with all specified requirements. Refer to the appendices and attachments for more details.
- 8. Submission Format:** Submit your source code file in the format
 <student number>.c , for example, 217123456.c
 (ONLY YOUR STUDENT NUMBER! Do not add any other text.)
- 9. Submission Upload:** Use the dedicated upload link on MyTUTorD2L to upload your C code only. While multiple submissions are allowed, only the latest submission will be retained. If you upload the wrong file by mistake, simply re-upload the correct one, and the previous submission will be overwritten.
- 10. Backup and Save:** Remember to save your work on the PC D:Drive and save regularly throughout the evaluation. In the event of PC malfunction or power failure, only 5 to 10 minutes (depending on the case) extra time will be allotted.

QUESTION:

Create a structured C program for a Parallelogram Management Menu. See Figure 1 below for the notation used.

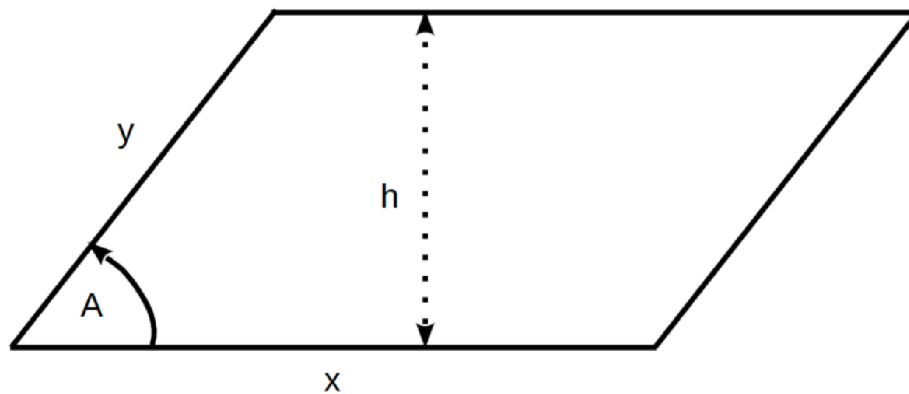


Figure 1: Parallelogram

x, y – side lengths of the parallelogram
A – angle between x and y (in degrees)
h – perpendicular height

Hint: Use the `asin()` function to calculate A in radians.

Requirements:

Before proceeding to the menu-driven section of the application, perform the following steps:

1. Prompt the user to enter a size for the dynamic arrays (x, y, h, A, surface)
2. Create dynamic arrays (x, y, h, A, surface) of the same size using 'malloc()' or 'calloc()', based on the size provided by the user. All arrays are of type int, except A, which is of type float.

```
Enter the number of parallelograms (positive integer > 1): |
```

Within the main function (after the dynamic memory allocation), a **switch statement** should be used to handle user selections. This switch statement should be nested within a loop, allowing the user to make multiple selections until they choose to exit.

Application functions to be implemented and corresponding user selections:

Function prototypes:

```
int displayMenu(void);
void generateRandomNumbers(int* x, int* y, int* h, float* A, int *surface, int size);
void displayArrays(int* x, int* y, int* h, float* A, int *surface, int size);
void sortDescending(int* x, int* y, int* h, float* A, int *surface, int size);
float findAve(int* h, int size);
int findMax(int* h, int size);
void linearSearchS(int* x, int* y, int* h, float* A, int *surface, int size, int value);
```

See the rubric at the end of the test for function mark allocations.

Function 1: DisplayMenu

Description:

This function displays the main menu options to the user and prompts them to make a choice. It then returns the user's choice as an integer.

Output:

```
=== Parallelogram Management Menu ===
1. Generate random parallelograms
2. Display arrays and surface area graph
3. Sort by x (descending) and display
4. Find average h value
5. Find maximum h value
6. Search for value in surface array (Linear Search)
7. Quit
Enter your choice (1-7): |
```

Details:

- The function does not take any parameters.
- It prints a list of menu options (numbered 1 - 7) using 'printf'.
- It uses 'scanf' to read the user's choice and stores it in a local variable 'choice'.
- The function returns the value of 'choice', which corresponds to the menu option selected by the user.

Function Prototype:

```
int DisplayMenu (void);
```

Example Call:

```
int choice = DisplayMenu();
```

Function 2: Generate Random Numbers**Description:**

This function generates random numbers for the x, y and h arrays. Populate the 'A' array with the formula: $\text{asin}\left(\frac{h}{y}\right) \times \left(\frac{180}{\pi}\right)$, where $\pi = 3.14159$. Populate the 'surface' array (surface area of each parallelogram) with the formula: $h \times x$.

Output:

```
Enter the number of parallelograms (positive integer > 1): 4

=== Parallelogram Management Menu ===
1. Generate random parallelograms
2. Display arrays and surface area graph
3. Sort by x (descending) and display
4. Find average h value
5. Find maximum h value
6. Search for value in surface array (Linear Search)
7. Quit
Enter your choice (1-7): 1
Random parallelograms generated.

Press Enter to continue...
```

Details:

- Parameters (refer to Figure 1):
 - 'x' (int*): An array of integers representing the x-side of the parallelograms
 - 'y' (int*): An array of integers representing the y-side of the parallelograms
 - 'h' (int*): An array of integers representing the vertical height of the parallelograms
 - 'A' (float*): An array of floats representing the angle between x and y (in degrees)
 - 'surface' (int*): An array of integers representing the surface area of each parallelogram
 - 'size' (int): The number of parallelograms
- The function iterates through each array index and generates a random number for:
 - x (15 – 30)

- y (15 – 30)
 - h (4 – 14)
- The function calculates the 'A' and 'surface' value to populate the 'A' and 'surface' arrays.

Function Prototype:

```
void generateRandomNumbers(int* x, int* y, int* h, float* A, int *surface, int size)
```

Example Call:

```
generateRandomNumbers(x, y, h, A, surface, size)
```

Function 3: Display Arrays and Graph**Description:**

This function displays a graphical representation of the 'surface' array. It prints all the values of the x, y, h and the A arrays followed by the graph.

Output:

```
=== Parallelogram Management Menu ===
1. Generate random parallelograms
2. Display arrays and surface area graph
3. Sort by x (descending) and display
4. Find average h value
5. Find maximum h value
6. Search for value in surface array (Linear Search)
7. Quit
Enter your choice (1-7): 2

Parallelograms:
Index  x      y      h      A
0      17     30     10     19.4712
1      23     22      5     13.1366
2      16     22      8     21.3237
3      18     16      6     22.0243

Surface Graph:
170: *****
115: *****
128: *****
108: *****

Press Enter to continue...
```

Details:

- Parameters (refer to Figure 1):
 - 'x' (int*): An array of integers representing the x-side of the parallelograms
 - 'y' (int*): An array of integers representing the y-side of the parallelograms
 - 'h' (int*): An array of integers representing the vertical height of the parallelograms
 - 'A' (float*): An array of floats representing the angle between x and y (in degrees)
 - 'surface' (int*): An array of integers representing the surface area of each parallelogram
 - 'size' (int): The number of parallelograms
- The function prints the values of each array.
- The function prints stars (*) equal to 'surface'/10. So, there is only one * for every 10 units of surface, for example if surface = 125, then print 12 stars.

Function Prototype:

void displayArrays(int* x, int* y, int* h, float* A, int *surface, int size)

Example Call:

displayArrays(x, y, h, A, surface, size)

Function 4: SortDescending**Description:**

This function sorts the 'x' array in descending order, highest to lowest, then displays the arrays and the 'surface' graph. The y, h, A and 'surface' arrays must also be sorted based on x.

Output:

```
=== Parallelogram Management Menu ===
1. Generate random parallelograms
2. Display arrays and surface area graph
3. Sort by x (descending) and display
4. Find average h value
5. Find maximum h value
6. Search for value in surface array (Linear Search)
7. Quit
Enter your choice (1-7): 3
Arrays sorted in descending order by x.

Parallelograms:
Index  x      y      h      A
0      23     22      5     13.1366
1      18     16      6     22.0243
2      17     30     10     19.4712
3      16     22      8     21.3237

Surface Graph:
115: *****
108: *****
170: *****
128: *****

Press Enter to continue...
```

Details:

- Parameters (refer to Figure 1):
 - 'x' (int*): An array of integers representing the x-side of the parallelograms
 - 'y' (int*): An array of integers representing the y-side of the parallelograms
 - 'h' (int*): An array of integers representing the vertical height of the parallelograms
 - 'A' (float*): An array of floats representing the angle between x and y (in degrees)
 - 'surface' (int*): An array of integers representing the surface area of each parallelogram
 - 'size' (int): The number of parallelograms
- The function uses nested loops to repeatedly compare and swap elements to sort the arrays.
- Then call the 'displayArrays' function to display the sorted arrays.

Function Prototype:

```
void sortDescending(int* x, int* y, int* h, float* A, int *surface, int size)
```

Example Call:

```
sortDescending(x, y, h, A, surface, size)
```

Function 5: Find Average**Description:**

This function calculates the average value of the 'h' array and returns it. The main program then prints the returned value.

Output:

```
=== Parallelogram Management Menu ===
1. Generate random parallelograms
2. Display arrays and surface area graph
3. Sort by x (descending) and display
4. Find average h value
5. Find maximum h value
6. Search for value in surface array (Linear Search)
7. Quit
Enter your choice (1-7): 4
Average h value: 7.2500

Press Enter to continue...
```

Details:

- Parameters (refer to Figure 1):
 - 'h' (int*): An array of integers representing the vertical height of the parallelograms
 - 'size' (int): The number of parallelograms
- The function uses a loop to calculate the average value of the 'h' array.
- It then returns the average value.

Function Prototype:

```
float findAve(int* h, int size)
```

Example Call:

```
a = findAve(h, size)
```

Function 6: Search Max Value

Description:

This function searches the 'h' array for the highest value and returns it. The main program then prints the returned value.

Output:

```
=== Parallelogram Management Menu ===
1. Generate random parallelograms
2. Display arrays and surface area graph
3. Sort by x (descending) and display
4. Find average h value
5. Find maximum h value
6. Search for value in surface array (Linear Search)
7. Quit
Enter your choice (1-7): 5
Maximum h value: 10

Press Enter to continue...
```

Details:

- Parameters (refer to Figure 1):
 - 'h' (int*): An array of integers representing the vertical height of the parallelograms
 - 'size' (int): The number of parallelograms
- The function initialises 'high' to the first element of 'h' and 'index' to 0.
- It iterates through the array to find the largest value and updates 'high' accordingly.
- The function returns the largest value of the 'h' array.

Function Prototype:

```
void findMax(int* h, int size)
```

Example Call:

```
m = findMax(h, size)
```

Function 7: LinearSearch

Description:

Use the Linear Search algorithm to find all positions where the entered value occurs in the 'surface' array. Then prints all the array values at that position.

Output:

```
=== Parallelogram Management Menu ===
1. Generate random parallelograms
2. Display arrays and surface area graph
3. Sort by x (descending) and display
4. Find average h value
5. Find maximum h value
6. Search for value in surface array (Linear Search)
7. Quit
Enter your choice (1-7): 6
Enter value to search in surface array: (> 0) 170
Positions of 170 in surface array:
Index=2, x=17, y=30, h=10 A=19.4712, surface = 170
Press Enter to continue...
```

```
=== Parallelogram Management Menu ===
1. Generate random parallelograms
2. Display arrays and surface area graph
3. Sort by x (descending) and display
4. Find average h value
5. Find maximum h value
6. Search for value in surface array (Linear Search)
7. Quit
Enter your choice (1-7): 6
Enter value to search in surface array: (> 0) 0
Enter value to search in surface array: (> 0) 250
Positions of 250 in surface array:
Value 250 not found in surface array.
Press Enter to continue...
```

Details:

- Parameters (refer to Figure 1):
 - 'x' (int*): An array of integers representing the x-side of the parallelograms
 - 'y' (int*): An array of integers representing the y-side of the parallelograms
 - 'h' (int*): An array of integers representing the vertical height of the parallelograms
 - 'A' (float*): An array of floats representing the angle between x and y (in degrees)
 - 'surface' (int*): An array of integers representing the surface area of each parallelogram
 - 'size' (int): The number of parallelograms
 - 'value' (int): The value that the user wants to search for.
- The function iterates through the 'surface' array to find a match for 'value'.

- If a match is found, it prints the index and the value of all arrays at the found index.
- If no match is found, it prints a message that no value is found.

Function Prototype:

```
void linearSearchS(int* x, int* y, int* h, float* A, int *surface, int size, int value)
```

Example Call:

```
linearSearchS(x, y, h, A, surface, size, value)
```

Ensure all functions mentioned above are self-written and listed in the specified order below the main function, following the function prototypes. Avoid the use of global variables, apply `system("cls")` and `getch()` for user-friendliness, and maintain clear indentation and comments for readability.

Implement The Following Flow Format:

1. Libraries
2. Function prototypes (1 to 7) - Given
3. Main function
 - A. Variables
 - B. Create Dynamic List
 - C. Main Application Menu
 - i. Do While
 1. Switch Statement
 - D. Free Dynamic Memory
 - E. Return 0
4. Function Implementation
 1. Display Menu Function
 2. Generate Random Numbers Function
 3. Display Arrays and Graph Function
 4. Sort Descending Function
 5. Average Function
 6. Max Function
 7. Linear Search Function

Adherence to the provided instructions, function naming conventions, and structured programming principles is crucial for full marks, even if the program functions correctly. Proper indentation and comments are also essential for clarity and understanding.

ANNEXURE A – MARK ALLOCATION

Note: Score range is 0 - 4 which is: 0-none, 1-poor, 2-average, 3-good, 4-excellent

TEST RUBRIC	SCORE [0-4]	WEIGHT [%]	MARK [%]
C CODE & OUTPUT EVALUATION		65	0
Overall Neatness, Indentation, and Spacing		4	0
User Defined Dynamic Arrays allocation and error checking		6	0
Free Dynamic Memory		6	0
2. Generate random numbers and calculate other values		8	0
3. Display arrays and surface area graph		10	0
4. Sort arrays in descending order by x		8	0
5. Find average h		5	0
6. Find maximum h		5	0
7. Linear search in surface array		8	0
No Runtime or Compile Errors		5	0
			0
			0
			0
TOTAL		65	0
STUDENT NUMBER			0

Graduate Attribute	GA Number	GA Score [0-5]
Engineering Professionalism	GA10	
Application of scientific and engineering knowledge	GA2	
Engineering methods, skills, tools, including information technology	GA5	
Impact of Engineering Activity	GA7	

ANNEXURE B – INFORMATION SHEET

Libraries:	<stdio.h> , <stdlib.h> , <time.h> , <math.h>
Data types:	void, char, short, int, float, double
Data Type modifiers:	const, auto, static, unsigned, signed
Arithmetic operators:	* / % + -
Relational operators:	< <= > >= == !=
Assignment operator:	= += -= *= /= %= &= ^= = <<= >>=
Logic operators:	&& !
Bitwise logic operators:	& ^ ~ << >>
Pointer operators:	Dereference: * Address: &
Control Structures:	
IF Selection:	if (condition) { ... };
IF ELSE Selection:	if (condition) { ... } else { ... };
SWITCH Selection:	switch (control variable) { case 'value': ... ; break; default: ... ; break; }
FOR Loop:	for (initial value of control variable; loop condition; increment of control variable) { ... }
WHILE Loop:	while (condition) { ... };
DO WHILE loop:	do { ... } while (condition);
Functions:	return_data_type function_name (parameters) { ... };
Common Library Functions:	printf() , scanf() , rand() , srand() , time() , isalpha() , isdigit() , getchar() , getch() , strcpy()
Arrays:	One dimensional: data_type variable_name[size]; Two dimensional: data_type variable_name [x_size][y_size];

ANNEXURE C – ASCII TABLE

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL